Last class on Trees →

1) Next pointer binary Tree ← O(N) ✓
O(1) ✓

2) Equal partition.

3) Largest BST subtree

→ LCA / Common node in BST

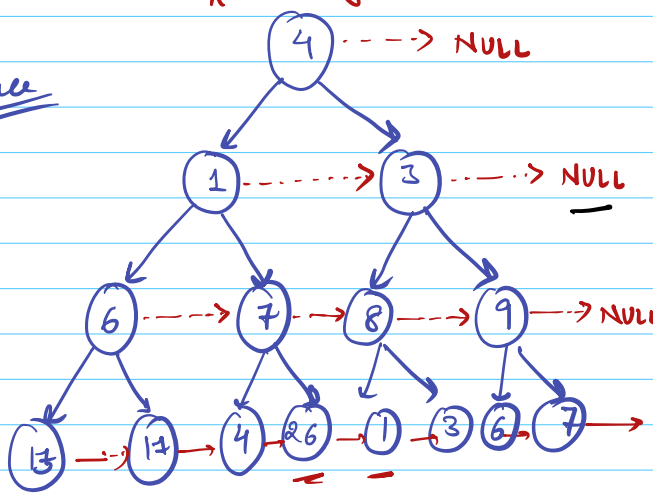
All tree concepts.
At last → If time LRU cache

Every level is completely filled

Perfect

. O(1) space

Q. Next pointer / binary Tree.



O(1) space

Given : Root of a BT

```
-> val
-> left
-> right
-> next
```

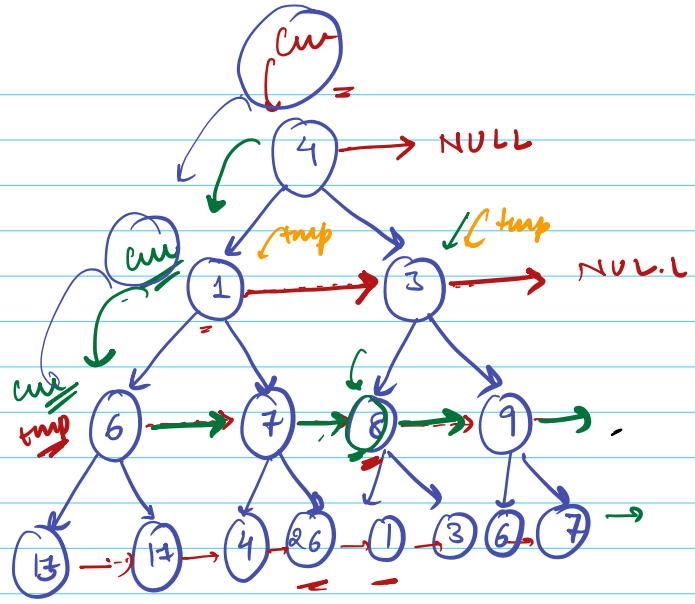
O(N)

Level order Traversal =>

```
qu.push(root)
qu.push(NULL)

while(!qu.empty())

    curr = qu.front()
    qu.pop()

    if(curr != null){
        curr.next = qu.front()
    }
    else if(curr == NULL and !qu.empty())
        qu.push(NULL) continue;
    if(curr.left) qu.push(curr.left)
    if(curr.right) qu.push(curr.right)
}
```

O(N) Space

O(1) space



13 → 17
17 → 4

```
6.left.next = 6.right
6.right.next = 6.next.left
```

tmp = tmp.next

4 → 26
26 → 1

```
7.left.next = 7.right
7.right.next = 7.next.left
```

tmp = tmp.next

SC O(1)

TC O(N)

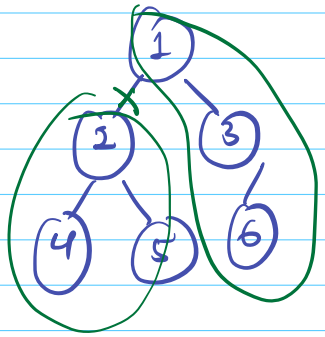
```
cur = root.
while (cur)
    tmp = cur
    while (tmp)
        tmp.left.next = tmp.right
        if (tmp.next)
            tmp.right.next = tmp.next.left
        tmp = tmp.next
```

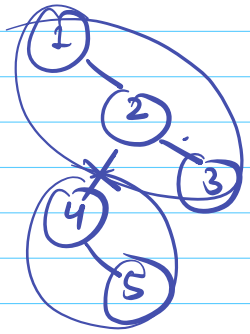
cur = cur . left

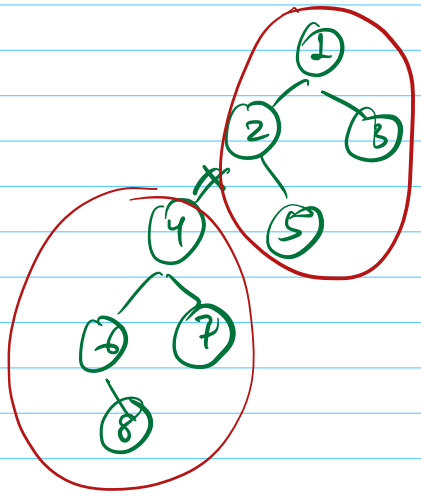
Q2. Equal Partition

You are given a binary Tree.

You can delete an edge !

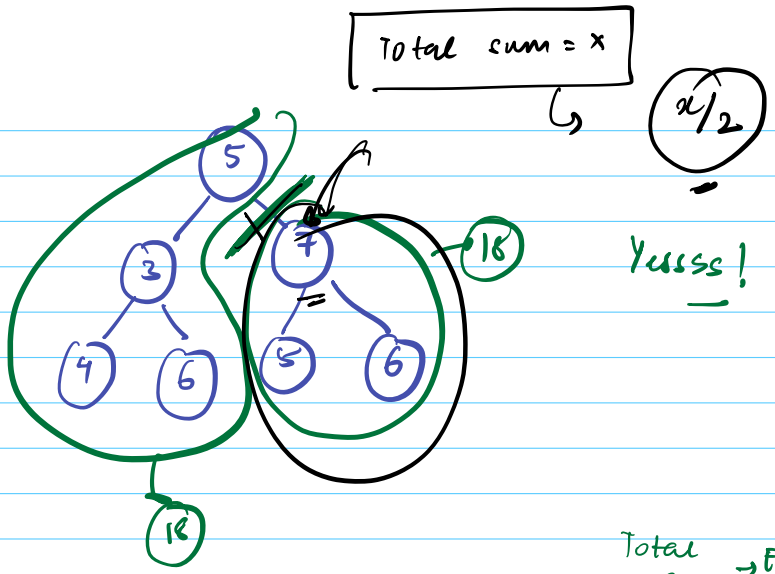




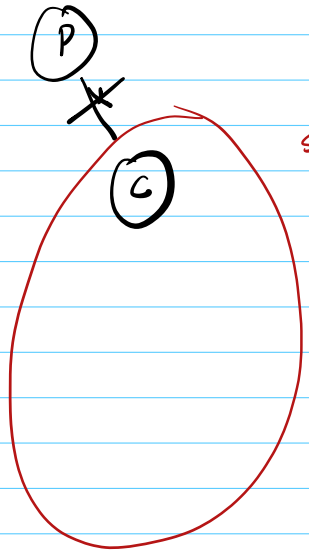


2 subtrees

Is it possible to delete edge s.t 2 B.T formed have equal sum. ?



Total sum → Even



Subtree rooted at child node

find

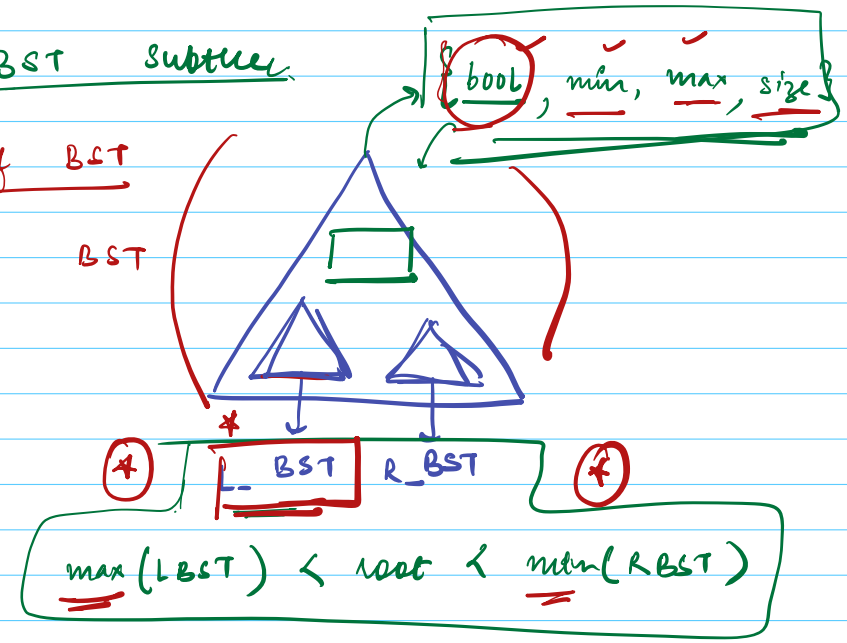
If A subtree with sum $x/2$? $O(N)$?

ans = 1

```
int sum( root )

  left
  right
  if ( L+R+ root.val == x/2 )
      ans = 1
```

3) Largest BST Subtree

Properties of BST



boolean ⇒ if my Left Subtree is a BST

min / max ⇒ root > max / root < min

size ⇒ ($l_s + R_s + 1$)

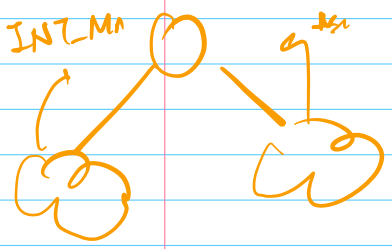
class (Ret) ⇒ { }

Ret isBST (root)

if (root == NULL) {

return { True, INT_MAX, INT_MIN }
(min) (max)

root < MIN



RetL = isBST (root.left)

RetR = isBST (root.right)

if (RetL.bool & RetR.bool)

if (RetL.max <= root.val
and RetR.min >= root.val)

return (True,

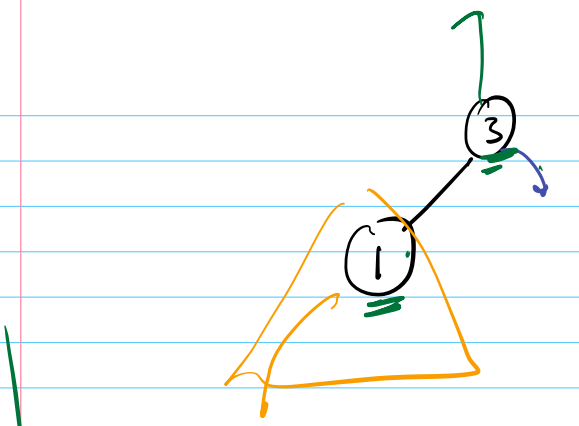
min(RetL.min,
root.val)

max (RetR.max,
root.val)

RetL.size + RetR.size) }

else

return { False, }



isBST(2)

↳ isBST(1)

↳ isBST(NULL)

1

True

min : INT_MAX

max : INT_MIN

size : 0

True

1

3

2

True

min(L.min, 1) 1

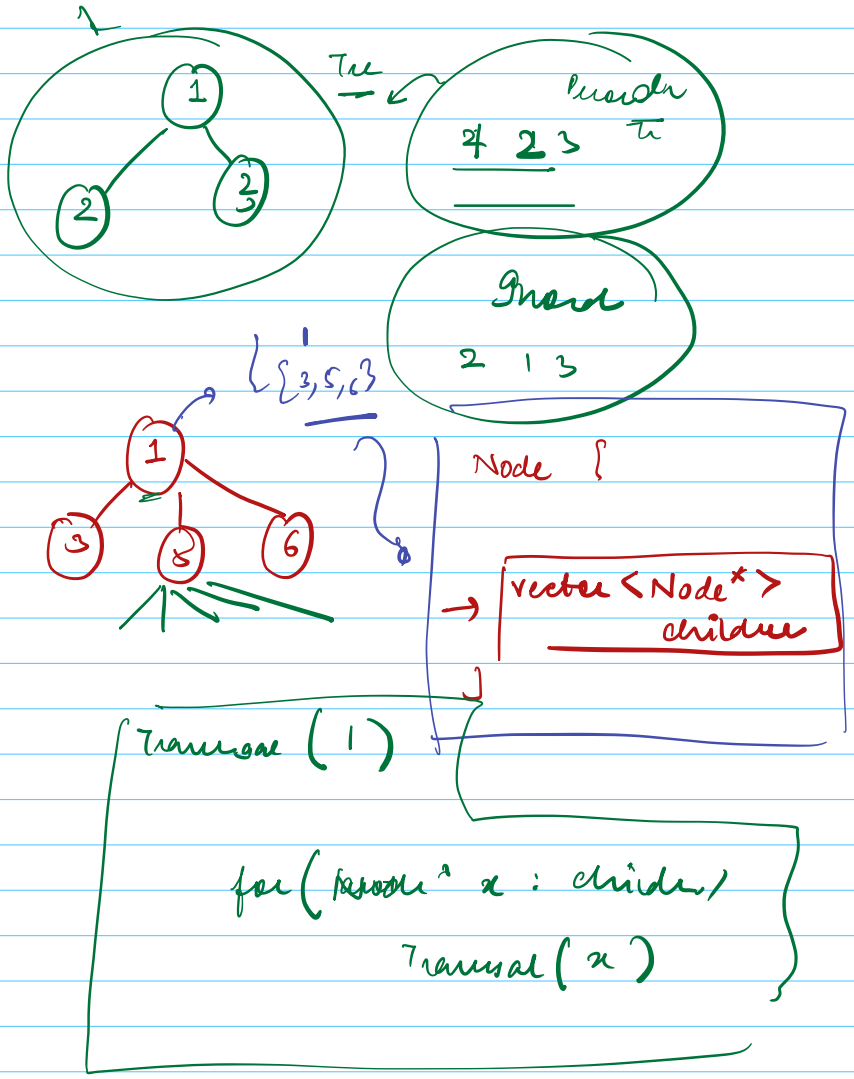
1

1

if (L.max < Root < R.min)

INT_MAX

INT_MIN


Tree
preorder
4 2 3
Graph
2 1 3
{{3,5,6}
Node {
vector<Node*>
children
Traversal(1)
for(Node* x : children)
Traversal(x)